

COMPUTER SCIENCE TRIPOS Part IB 75%, Part II 50% – 2021 – Paper 7

10 Prolog (acr31)

A *Caesar Cipher* (or Shift Cipher) produces ciphertext from plaintext by replacing each letter with another that is found a fixed number of places down the alphabet. Users provide a key from 1 to 25 (inclusive) to determine the number of places to move. Our alphabet contains just the 26 lowercase letters and is circular: moving past **z** takes you back round to **a** again. For example under a key of 5 the letter **y** would be replaced by the letter **d**.

When answering this question ensure that each of your predicates has a comment giving a declarative reading of its behaviour and avoid unnecessary use of cut. Do not use any extra-logical predicates (such as `assertz`) or any library predicates.

terms, ground
terms, predicates

- (a) One way to represent the ordering of characters is with 26 facts indicating the next character. For example `next(a,b)` then `next(b,c)` through to `next(z,a)`.

Use `next` to implement a predicate `nextn(N,C1,C2)` which succeeds if the character `C2` appears `N` places after the character `C1`. You may assume that `N` is always a ground term. [3 marks]

Answer:

```
% the zeroth character following C1 is C1, otherwise given CN that
% follows C1, the Nth character following C1 is the N-1th character
% following CN.
nextn(0,C1,C1).
nextn(N,C1,C2) :- N>0, N1 is N-1, next(C1,CN), nextn(N1,CN,C2).
```

Base case (1 mark), Recursive case (1 mark), Declarative explanation (1 mark). Candidates may either include the condition `N>0` or use a cut on the base case. Deduct one mark from candidates who fail to deliver correct backtracking behaviour here if they also fail to correct it in part (d) below.

declarative
explanation of
predicates, lists

- (b) Another approach would be to use a list of characters to record the order of letters.

Provide an alternative implementation of `nextn` which makes use of the list representation `[a,b,c,...]`. Explain how you deal with the case of moving past the end of the alphabet.

You may assume the existence of two predicates: `scan(C,R,List)` which succeeds if `R` is the remainder of `List` that follows the letter `C`; and `charAt(N,C1,List)` which succeeds if `C1` is the character at position `N` in `List`. Position 0 is the first element of the list. `N` must be a ground term. [6 marks]

Answer:

```
scan(C,R,[C|R]).
scan(C,R,[_|T]) :- scan(C,R,T).
```

```
charAt(0,C,[C|_]).
charAt(N,C,[_|T]) :- N>0, N1 is N-1, charAt(N1,C,T).
```

```
alphabet([a,b,c,...,y,z,a,b,c,...,y]). % (you don't need the final z)
```

```
% nextn succeeds if the Nth item in the remainder of the alphabet
% that begins with the letter C1 is C2.
```

```
nextn(N,C1,C2) :- alphabet(Alpha),
                    scan(C1,Rest,Alpha),
                    charAt(N,C2,[C1|Rest]).
```

3 marks for code that works to a large extent e.g. the circular alphabet is dealt with by including each letter twice in the list (2 of the 3 marks) and avoiding an off-by-one error: the first letter of `Rest` after `scan` is 1 step from the original letter, not 0. (1 of the 3 marks). Declarative comment for `nextn` (1 mark). Some way of showing they are using a list of letters for ordering (1 mark). Using `scan` and then `charAt` in some way (1 mark).

Other methods of dealing with the circular alphabet and avoiding the off-by-one error are acceptable. Examples include: rotating the list until the head element is the one needed, finding the position of both characters and using modular arithmetic. Passing a non-ground term at position 0 of `charAt` is not acceptable.

data
representation

- (c) Compare the merits of these two representations giving three relative benefits or drawbacks. [3 marks]

Answer: From a practical perspective the code for the facts representation is simpler: 26 facts and a single predicate vs. 3 predicates (1 mark). The facts also capture the idea of the circular alphabet which is not natural in the list form (1 mark). However, depending on the way that the Prolog interpreter searches for facts the list representation could be more efficient: the list form does a single pass through the list whereas the facts representation has to potentially scan all the 'next' facts for each step through the alphabet looking for the next letter. (1 mark) Alternatively, the compiler might index facts efficiently and be able to resolve each one in constant time.

Other valid arguments will be given credit. Arguments comparing aspects of the *implementation* which are not fundamental to the *representation* (e.g. an inefficient implementation where a more efficient approach is possible) are not acceptable.

facts, rules

- (d) Implement a predicate `caesar(K,P,C)` which succeeds if `C` is the ciphertext of the plaintext `P` under key `K`. Both ciphertext and plaintext are represented with a list of letters. You may assume that `P` and `K` are ground terms. [3 marks]

Answer:

```
% the cipher of the empty list is empty, otherwise the cipher of a list with
% head H and tail T is the K character shift of H with the cipher of T.
caesar(_,[],[]).
caesar(K,[H|T],[R|S]) :- nextn(K,H,R),caesar(K,T,S).
```

Declarative comment (1 mark), Base case for empty list (1 mark), Recursive case using `nextn` (1 mark).

iterative
deepening,
restrictions on
arithmetic

- (e) The plaintext for some of the ciphertext characters has been discovered through a known-plaintext attack. Extend your `caesar` predicate to recover the key in this scenario and give an example invocation. [5 marks]

Answer:

```
range(Min,_,Min).  
range(Min,Max,Ans) :- N2 is Min+1, N2 < Max, range(N2,Max,Ans).
```

```
% the cipher of the empty list is empty, otherwise the cipher of a list with  
% head H and tail T is the K character shift of H with the cipher of T where  
% K is drawn from the numbers 1 to 25.
```

```
caesar(_,[],[]).  
caesar(K,[H|T],[R|S]) :- range(0,26,K),nextn(K,H,R),caesar(K,T,S).
```

```
% example invocation  
:- caesar(K,[g,_,_,_],[h,e,a,c]).
```

The principal observation here is that `nextn` requires a ground term for its first argument and so you can't just use the previous implementation with an unbound key. The solution shown here is iterative deepening where we try all Keys starting with 1 by drawing from a range. Another option would be to draw them from a list of `[1,...25]`.

Declarative comment (1 mark), example invocation (1 mark), observation that you need to ground K (1 mark), iterative deepening strategy (2 marks).

Alternative solutions are also acceptable.
